

```

# Load required libraries
library(kohonen)
library(ggplot2)

# Read the data
credit_data <- read.csv("creditworthiness.csv")

# Assign proper column names
col_names <- c(
  "functionary", "rebalanced_overdrawn", "FI3O_credit_score", "gender",
  "accounts_other_banks", "credit_refused_past", "years_employed",
  "savings_other_accounts", "self_employed",
  "max_balance_12m", "min_balance_12m", "avg_balance_12m",
  "max_balance_11m", "min_balance_11m", "avg_balance_11m",
  "max_balance_10m", "min_balance_10m", "avg_balance_10m",
  "max_balance_9m", "min_balance_9m", "avg_balance_9m",
  "max_balance_8m", "min_balance_8m", "avg_balance_8m",
  "max_balance_7m", "min_balance_7m", "avg_balance_7m",
  "max_balance_6m", "min_balance_6m", "avg_balance_6m",
  "max_balance_5m", "min_balance_5m", "avg_balance_5m",
  "max_balance_4m", "min_balance_4m", "avg_balance_4m",
  "max_balance_3m", "min_balance_3m", "avg_balance_3m",
  "max_balance_2m", "min_balance_2m", "avg_balance_2m",
  "max_balance_1m", "min_balance_1m", "avg_balance_1m", "credit_rating"
)

names(credit_data) <- col_names

# Basic statistical analysis
print("Dataset structure:")
str(credit_data)

# Count the number of assessed and unassessed clients
print("Credit rating distribution:")
print(table(credit_data$credit_rating))

# Separate the assessed data for analysis
assessed_data <- credit_data[credit_data$credit_rating != 0, ]
print("Credit rating distribution among assessed clients:")
print(table(assessed_data$credit_rating))

# Calculate correlations with target
corr_values <- numeric(46)
for (i in 1:46) {
  corr_values[i] <- abs(cor(assessed_data[,i], assessed_data$credit_rating,
    use="complete.obs"))
}

```

```

# Create a data frame with feature names and correlations
feature_corr <- data.frame(
  feature = col_names[1:46],
  correlation = corr_values)

# Sort by correlation (descending)
feature_corr <- feature_corr[order(-feature_corr$correlation),]

# Get top 5 feature names
top5_names <- feature_corr$feature[1:5]

# Prepare data for visualization
# Convert target to factor for better visualization
assessed_data$credit_rating_factor <- factor(assessed_data$credit_rating,
  levels = c(1, 2, 3),
  labels = c("A", "B", "C"))

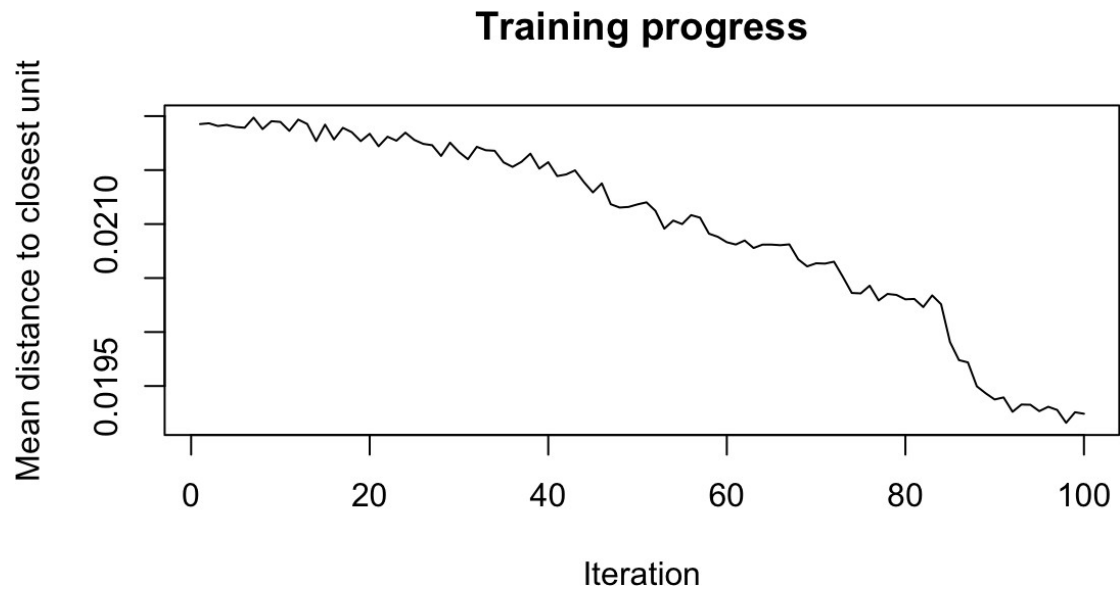
# Create distribution plots for top 5 attributes by credit rating
par(mfrow = c(3, 2))
for (attr in top5_names) {
  # Check if column exists
  if(attr %in% names(assessed_data)) {
    boxplot(assessed_data[[attr]] ~ assessed_data$credit_rating_factor,
      main = paste("Distribution of", attr, "by Credit Rating"),
      xlab = "Credit Rating", ylab = attr)
  } else {
    print(paste("ERROR: Column", attr, "not found in dataset"))
  }
}

# SOM
# Normalize the data for SOM
data_for_som <- scale(assessed_data[, 1:46])

# Train the SOM
set.seed(123)
som_grid <- somgrid(xdim = 10, ydim = 10, topo = "hexagonal")
som_model <- som(data_for_som, grid = som_grid, rlen = 100)

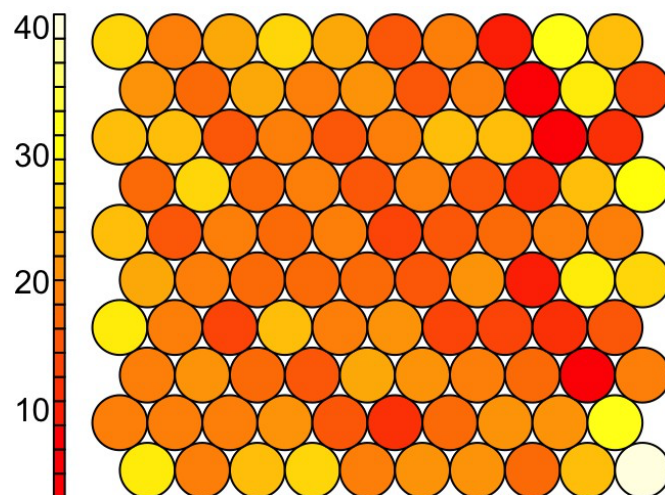
# Plot the SOM results
plot(som_model, type = "changes")

```



```
plot(som_model, type = "count")
```

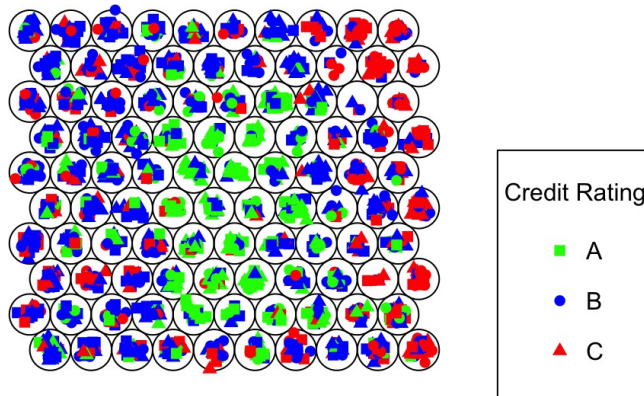
Counts plot



```
# Create a mapping plot colored by credit rating
plot(som_model, type = "mapping",
     pchs = 15:17,
     col = c("green", "blue", "red")[assessed_data$credit_rating],
     main = "SOM Mapping by Credit Rating")
```

```
# Add a legend
legend("bottomright",
     legend = c("A", "B", "C"),
     pch = 15:17,
     col = c("green", "blue", "red"),
     title = "Credit Rating")
```

SOM Mapping by Credit Rating

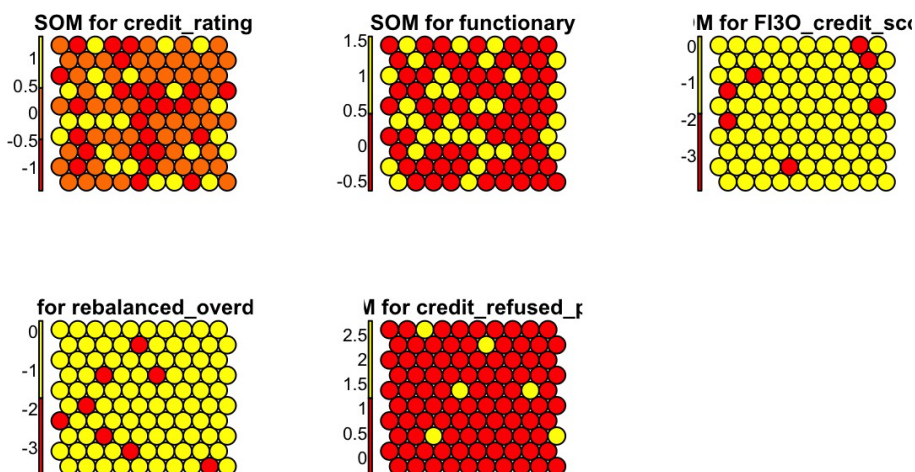


Quality analysis of the SOM

```
som_quality <- sum(som_model$distances) / nrow(assessed_data)
cat("Average distance to BMU:", som_quality, "\n")
```

Create a property plot for the SOM for each top feature

```
par(mfrow = c(2, 3))
for (i in 1:length(top5_names)) {
  plot(som_model, type = "property",
       property = data_for_som[, top5_names[i]],
       main = paste("SOM for", top5_names[i]))
}
```



Check class distribution in each node to assess separation

```
som_node_classes <- table(som_model$unit.classif, assessed_data$credit_rating)
pure_nodes <- sum(rowSums(som_node_classes > 0) == 1)
mixed_nodes <- sum(rowSums(som_node_classes > 0) > 1)
cat("Pure nodes (only one class):", pure_nodes, "\n")
```

```
cat("Mixed nodes (multiple classes):", mixed_nodes, "\n")
cat("Percentage of pure nodes:", round(pure_nodes / (pure_nodes + mixed_nodes) * 100,
2), "%\n")
```

```
library(RSNNS)
```

```
# Select all entries for which credit rating is known
```

```
knownData <- subset(credit_data, credit_data[,46] > 0)
```

```
# Select all entries for which credit rating is unknown
```

```
unknownData <- subset(credit_data, credit_data[,46] == 0)
```

```
# Separate value from targets
```

```
trainValues <- knownData[, 1:45] #select all rows of the first 45 columns
```

```
# DecodeClassLabels() convert input to a binary matrix
```

```
trainTargets <- decodeClassLabels(knownData[,46])
```

```
# Select the credit rating column as the target
```

```
unknownValues <- unknownData[, 1:45]
```

```
# Split data into training and test sets
```

```
trainset <- splitForTrainingAndTest(trainValues, trainTargets, ratio=0.5)
```

```
#normTrainingAndTestSet: the default is to not normalize targets
```

```
#normalizing: standardization of scale,
```

```
trainset <- normTrainingAndTestSet(trainset)
```

```
# Build MLP
```

```
model <- mlp(trainset$inputsTrain, trainset$targetsTrain, size=5, maxit=250,
```

```
learnFuncParams = c(0.01), inputsTest = trainset$inputsTest, targetsTest =
```

```
trainset$targetsTest)
```

```
predictTestSet <- predict(model, trainset$inputsTest)
```

```
# Construct confusion matrix, a matrix of how well a classification model is performing
```

```
#by comparing the predicted class labels to the actual class labels
```

```
confusionMatrix(trainset$targetsTrain, fitted.values(model))
```

```

      predictions
targets  1    2    3
1 132   83   11
2  36  436   31
3  15   78 159

```

```
confusionMatrix(trainset$targetsTest, predictTestSet)
```

	predictions		
targets	1	2	3
1	108	126	23
2	72	340	55
3	26	132	99